MSc ARTIFICIAL INTELLIGENCE AND COMPUTER SCIENCE

Dissertation Report

Predictive Maintenance for Industrial Motors:

A Cascaded Machine Learning Pipeline for Fault Detection, Severity Staging, Fault Diagnosis and Maintenance Decision Support

Using ESP32, MPU-6500, DHT-11, Voltage Sensor and Multi-Stage Supervised Learning

Supervised by: Prof. Panagiotis Andriotis

Inspected by: Prof. Kashif Rajpoot

Author: Marina Ramis

Program: MSc Artificial Intelligence and Computer Science

GitLab Repository: https://git.cs.bham.ac.uk/projects-2025-26/myr599#

Submission Date: September 2026

Abstract

Unplanned failure of industrial motors is a significant source of production loss, safety risk and unbudgeted repair expenditure across manufacturing, oil and gas, mining and utility sectors. This dissertation presents the design, implementation and evaluation of a cascaded machine learning pipeline for condition-based predictive maintenance of industrial motors, built on a simulated Industrial Internet of Things (IIoT) sensor stream inspired by an ESP32 microcontroller coupled to an MPU-6500 inertial measurement unit, together with temperature, voltage and current sensors.

The pipeline is organized into four sequential decision stages rather than a single classifier. Module 1 performs binary health screening using a k-Nearest Neighbors (KNN) classifier to separate healthy from faulty operating states. Module 2 estimates a five-level health severity state (healthy, degraded, warning, critical, fault) using a Random Forest classifier, allowing maintenance teams to reason about the trajectory of degradation rather than a single healthy or faulty label. Module 3 identifies the underlying root cause of a fault among five categories (bearing wear, imbalance, overload, phase loss and insulation failure) using an XGBoost classifier trained on frequency of vibration, current, voltage and temperature features. Module 4 consumes the outputs of Modules 2 and 3 and joins them against a maintenance knowledge base to generate a prioritized maintenance action, an estimated repair cost, an estimated downtime and a remaining useful life indicator.

The dataset comprises 4,556 five-second sensor readings across 36 engineered and raw features, with no missing values or duplicate rows. Careful attention was paid to data leakage: the operational load factor, an engineered power feature exhibiting near-collinearity with motor current, and an ambient temperature feature confirmed to carry no discriminative signal, were excluded from the model input space, and this decision is justified quantitatively in the methodology chapter rather than asserted.

On the held-out evaluation partitions, the Random Forest severity classifier and the XGBoost fault-type classifier achieved perfect classification accuracy on both the internal test split and an independently generated deployment test set, while the binary KNN health detector achieved an accuracy of 88.5% and a ROC-AUC of 0.994 on its reported evaluation snapshot. The maintenance recommendation stage translated 128 held-out predictions into a projected total remediation cost of 25,400 US dollars and 220 hours of

downtime across the deployment window, illustrating how the cascade converts a classification output into an actionable, costed maintenance plan.

The principal contribution of this work is architectural rather than purely algorithmic: while individual fault-classification models are well represented in the literature, this dissertation demonstrates and critically evaluates an end-to-end cascade that couples binary detection, severity staging, root-cause diagnosis and automated, costed maintenance recommendation within a single reproducible pipeline, and it explicitly interrogates the accuracy obtained on the later stages as a methodological finding in its own right rather than an unqualified success. The report closes with a discussion of the limitations imposed using simulated data, the implications of the observed ceiling effect on later modules, and a set of concrete directions for validating the pipeline against noisier, real-world instrumentation.

Honour Code **"Declaration of AI-Assisted Work"**

"I certify that this project is my own work. Research and partial coding were assisted by ChatGPT 4.0. The report structure and initial drafts are mine; final editing was assisted by Claude Sonnet for clarity and consistency. All outputs have been reviewed and edited to accurately reflect my work."

Table of Contents

Abstract ... 2
Table of Contents ... 4
1.1 Background and Motivation ... 6
1.2 Problem Statement ... 7
1.3 Aim and Objectives ... 7
1.4 Scope and Assumptions ... 8
1.5 Report Structure ... 8
2. Novel Contribution and Positioning Against Existing Work ... 9
2.1 What Is Not Claimed as Novel ... 9
2.2 What Is Claimed as Novel ... 9
2.2.1 A four-stage decision cascade rather than a single classifier ... 9
2.2.2 Justified data-leakage screening ... 10
2.2.3 Coupling diagnosis to a costed maintenance knowledge base ... 10
2.2.4 Critical evaluation of accuracy ... 10
3. Dataset and Exploratory Data Analysis ... 11
3.1 Data Acquisition Architecture ... 11
3.2 Dataset Description ... 11
3.3 Target Label Distributions ... 11
3.3.1 Structural relationship between fault type and health state ... 13
3.4 Feature Screening and Data Leakage Analysis ... 14
3.4.1 Operational load factor ... 14
3.4.2 Motor power ... 14
3.4.3 Ambient temperature ... 14
3.5 Distribution of Key Condition Indicators ... 15
3.6 Summary of Relevant Modelling Findings ... 16
4. System Architecture and Methodology ... 17
4.1 Overall Pipeline Architecture ... 17
4.2 Data Splitting and Evaluation Strategy ... 17
4.3 Module 1: Binary Health Detection ... 18
4.3.1 Problem framing ... 18
4.3.2 Algorithm justification ... 18
4.3.3 Hyperparameter selection ... 19
4.4 Module 2: Health Severity Classification ... 19
4.4.1 Problem framing ... 19
4.4.2 Algorithm justification ... 19
4.4.3 Hyperparameters ... 20

4.5 Module 3: Fault-Type Diagnosis 20
4.5.1 Problem framing 20
4.5.2 Algorithm justification 20
4.5.3 Hyperparameters 21
4.6 Module 4: Maintenance Recommendation Engine 21
4.6.1 Design 21
4.6.2 Design justification 21
4.7 Evaluation Metrics 22
5. Implementation 23
5.1 Software Stack 23
5.2 Codebase Structure 23
5.3 Feature Selection Implementation 24
5.4 Model Persistence and Deployment Pattern 24
5.5 Maintenance Recommendation Join 25
6. Results and Evaluation 26
6.1 Module 1: Binary Health Detection (KNN) 26
6.2 Module 2: Health Severity Classification (Random Forest) 28
6.3 Module 3: Fault-Type Diagnosis (XGBoost) 30
6.4 Module 4: Maintenance Recommendation Engine 31
6.5 Cross-Module Summary 31
7. Physical Hardware Implementation 33
8. Conclusion and Future Work 36
8.1 Summary of Contribution 36
8.2 Future Work 36
8.3 Closing Remarks 37
Appendix A: Supporting Material 39
A.1 Model Hyperparameter Summary 39
A.2 Maintenance Knowledge Base 39
A.3 Reproducibility Notes 40
Appendix B: Code Repository 41
B.1 Repository URL 41
B.2 Repository Contents 41
B.3 Instructions to Run the Code 42
Appendix C: Physical Hardware Implementation Evidence 44
C.1 Temperature and Humidity Node (DHT11) 44
C.2 Inertial Measurement Node (MPU-6500) 45
C.3 Voltage Sensing Node 46
C.4 Note on Provenance 48

1. Introduction

1.1 Background and Motivation

Industrial motors are among the most widely deployed pieces of rotating machinery in modern industry, powering pumps, fans, compressors, conveyors and countless other assets across manufacturing, energy, water treatment and process industries. Traditional maintenance strategies fall broadly into two categories: reactive maintenance, in which repair is only undertaken after a failure has occurred, and preventive maintenance, in which components are serviced on a fixed calendar or usage schedule irrespective of their actual condition. Both strategies are known to be economically inefficient: reactive maintenance incurs unplanned downtime, secondary damage and expedited-repair costs, while preventive maintenance frequently replaces components that still have useful life remaining, and yet still fails to anticipate faults that develop between scheduled inspections.

Predictive maintenance (PdM) addresses these shortcomings by monitoring the real-time condition of an asset through sensor instrumentation and using that condition signal to schedule maintenance only when there is quantitative evidence that it is required. The growth of low-cost microcontrollers, inertial measurement units and industrial IoT connectivity have made continuous condition monitoring economically viable even for medium and small motors that would previously not have justified dedicated vibration-analysis instrumentation. At the same time, advances in supervised machine learning have made it possible to convert raw multi-sensor streams into actionable classifications without requiring a physics-based diagnostic model for every fault mechanism.

This dissertation is motivated by a practical observation; most published machine learning studies of motor condition monitoring frame the problem as a single classification task, typically either binary healthy-versus-faulty detection or multi-class fault-type identification performed in isolation. In an operational maintenance department, however, a single classification label is rarely sufficient. A maintenance planner needs to know not only that a motor is unhealthy, but how severe the degradation is, what is physically wrong with it, and what it will cost in money and downtime to intervene, so that the intervention can be prioritized against other assets competing for the same maintenance budget and technician hours. This project was therefore designed around a staged decision pipeline that mirrors how a maintenance engineer reasons about an asset, rather than around a single leaderboard metric.

1.2 Problem Statement

Given a stream of multi-sensor readings from an operating motor, comprising rotational speed, electrical current, voltage, temperature and triaxial accelerometer and gyroscope statistics, this project addresses the following coupled problem: (i) determine whether the motor is currently healthy or faulty; (ii) if unhealthy, quantify the severity of the condition on an ordinal scale; (iii) identify the most probable root-cause fault mechanism; and (iv) translate that diagnosis into a concrete, costed maintenance recommendation with an associated priority level and an estimate of remaining useful life. Each of the four sub-problems has different label semantics, different class balance characteristics and arguably benefit from a different learning algorithm, which motivates a modular, cascaded architecture rather than a single monolithic model.

1.3 Aim and Objectives

The aim of this project is to design, implement and critically evaluate an end-to-end, modular machine learning pipeline that supports predictive maintenance decision making for industrial motors, from raw sensor ingestion through to a costed maintenance recommendation. The specific objectives are to:

- design an Industrial IoT condition monitoring architecture and identify the sensor modalities required to characterize motor health;
- perform structured exploratory data analysis to characterize the dataset, verify data quality and identify features that are informative, redundant or subject to data leakage;
- develop and justify, on quantitative grounds, a binary health-detection model (Module 1);
- develop a multi-class health severity classifier able to distinguish healthy, degraded, warning, critical and fault states (Module 2);
- develop a multi-class fault-type classifier able to distinguish bearing wear, imbalance, overload, phase loss and insulation failure (Module 3);
- implement a rule-based maintenance recommendation engine that converts Module 2 and Module 3 outputs into a prioritized, costed maintenance action (Module 4);

- evaluate each module using appropriate, class balance aware metrics, and critically discuss the results, including their limitations, rather than reporting accuracy figures in isolation;
- position the resulting system relative to existing predictive maintenance literature and identify explicitly what is new about the approach taken here.

1.4 Scope and Assumptions

The dataset used throughout this dissertation is a simulated sensor stream coupled with measurements captured from a physical motor test rig, generated to approximate the statistical behavior of a real motor under a defined set of fault conditions and operating loads. This is an explicit and important scope boundary: the pipeline is evaluated on data whose generative process is known and whose class boundaries are, by construction, comparatively clean. The scope of the dissertation covers model development, offline evaluation and a deployment-style test against an independently drawn evaluation file.

1.5 Report Structure

The remainder of this report is organized as follows. Chapter 2 states what is novel about the work. Chapter 3 describes the dataset and the exploratory data analysis that informed feature engineering decisions. Chapter 4 sets out the system architecture and methodology for all four modules, including the justification for algorithm choice, feature selection and the evaluation strategy. Chapter 5 describes the implementation. Chapter 6 presents the evaluation results for each module together with a discussion of what those results mean.

2. Novel Contribution and Positioning Against Existing Work

A recurring criticism of machine learning projects is to apply a well-known algorithm to a dataset and report an accuracy figure without articulating what is new about the work relative to the substantial body of prior art. This chapter addresses that concern directly by stating, explicitly and separately, what is and is not novel about this dissertation.

2.1 What Is Not Claimed as Novel

In the interest of accurate self-assessment, it is important to state plainly that none of the three learning algorithms used in this project, namely k-Nearest Neighbors, Random Forest and XGBoost, are themselves novel; they are long established, widely documented supervised learning methods, and their application to motor condition monitoring specifically has substantial precedent. Likewise, the general concept of using multi-sensor IIoT data for predictive maintenance is well established, and the general practice of engineering statistical vibration features such as root-mean-square, peak and standard-deviation values from accelerometer and gyroscope signals is standard in the condition monitoring literature. This project does not claim to introduce a new algorithm, a new feature extraction technique, or a new sensor modality.

2.2 What Is Claimed as Novel

The contribution of this dissertation is architectural and evaluative rather than algorithmic, and rests on four specific points.

2.2.1 A four-stage decision cascade rather than a single classifier

This dissertation decomposes the problem into a cascade of four coupled but distinct decisions: binary health screening (Module 1), ordinal severity staging across five levels (Module 2), root cause fault-type diagnosis across five fault categories (Module 3), and translation of the diagnosis into a costed, prioritized maintenance recommendation with an associated remaining useful life estimate (Module 4). Each stage uses a label space, class balance and, in three cases, a learning algorithm suited to its own sub-problem, rather than forcing a single model to serve all four purposes.

2.2.2 Justified data-leakage screening

Rather than including every available column as a model feature, this project identifies and removes three features on stated, testable grounds: the operational load factor, which is shown in Chapter 4 to directly encode the severity label used to generate the simulated dataset and is therefore excluded as a leakage risk; motor power, which is shown to correlate with motor current and is therefore redundant rather than informative; and ambient temperature, which is shown to carry no statistically meaningful separation across health states and is therefore excluded as noise. This screening is documented as a reproducible step in the codebase (see DROP_COLS in Chapter 3), and its effect is quantified in Chapter 4, rather than being an implicit or undocumented preprocessing choice.

2.2.3 Coupling diagnosis to a costed maintenance knowledge base

Third, Module 4 couples the statistical output of Modules 2 and 3 to an external, editable maintenance knowledge base, producing a maintenance priority, a recommended corrective action, an estimated repair cost in US dollars, an estimated downtime in hours and a remaining useful life estimate in days for every prediction.

2.2.4 Critical evaluation of accuracy

Fourth, and reflecting explicit supervisory feedback received on this project, the evaluation in Chapter 6 does not simply report the high accuracy achieved by the severity and fault-type classifiers as a success, accuracy values approaching 1.00 may indicate either a genuinely separable task or an artefact of the data generation and evaluation process, this dissertation treats the perfect or near perfect test accuracy obtained by Modules 2 and 3 as a result requiring interpretation rather than celebration.

2.3 Summary

In summary, the novelty of this dissertation lies in the combination and sequencing of established techniques into an end-to-end cascade that mirrors real maintenance decision-making, in the quantitative justification of feature exclusion decisions, in the extension of the pipeline to a costed maintenance recommendation, and in the critical stance taken toward the pipeline's own results. These contributions are evaluative and architectural rather than algorithmic, and are presented as such throughout the remainder of this report.

3. Dataset and Exploratory Data Analysis

3.1 Data Acquisition Architecture

The conceptual data acquisition architecture underlying the dataset is an ESP32 microcontroller polling an MPU-6500 six-axis inertial measurement unit alongside dedicated temperature, voltage and current sensors attached to an industrial motor, at a nominal sampling interval of five seconds. The raw signal from the accelerometer and gyroscope is not retained sample by sample in the working dataset; instead, each five-second window is summarized into mean, root-mean-square (RMS), peak and standard-deviation statistics per axis.

3.2 Dataset Description

Volume and structure. The working dataset, sensor_data.csv, contains 4,556 observations across 36 columns. Data quality checks performed as part of the exploratory analysis (Data_EDA.py, Health_detection.py) confirmed zero missing values and zero duplicate rows across the entire dataset, which is an important precondition for the modelling decisions taken in Chapter 4.

Feature groups. The 36 columns fall into six groups: (i) a timestamp and three label columns (health_label, health_state, fault_type, plus an integer fault_code); (ii) one operational parameter, load_factor; (iii) four electrical measurements, motor_speed_rpm, motor_current_a, motor_voltage_v and motor_power_kw; (iv) three temperature measurements, temp_ambient_c, temp_surface_c and temp_bearing_c; (v) one vibration severity summary, vibration_severity_ms2, alongside motor_vibration_mm_s; and (vi) twenty-one MPU-6500-derived accelerometer and gyroscope statistics (mean, RMS, peak and standard deviation across the X, Y and Z axes for the accelerometer, and mean, RMS and peak for the gyroscope).

3.3 Target Label Distributions

Three target labels are present in the dataset, corresponding to the three classification stages of the pipeline. Table 3.1 summarizes the five-level health_state distribution, which is the target for Module 2, and Table 3.2 summarizes the six-level fault_type distribution,

which is the target for Module 3. Module 1's binary target is derived directly from health_state by mapping the healthy class to 0 and all four remaining states to 1.

Health state	Count	Proportion
Healthy	2,575	56.5%
Degraded	740	16.2%
Warning	532	11.7%
Critical	414	9.1%
Fault	295	6.5%

Table 3.1. Health state class distribution (n = 4,556).

Fault type	Count	Proportion
None	2,865	62.9%
bearing_wear	540	11.9%
Imbalance	450	9.9%
phase_loss	295	6.5%
Overload	232	5.1%
insulation_failure	174	3.8%

Table 3.2. Fault type class distribution (n = 4,556).

Both labels are moderately imbalanced, with the majority class (healthy / none) accounting for approximately 57% to 63% of records and the rarest class (fault / insulation_failure) accounting for under 7%. This imbalance is a material consideration for model evaluation and is one of the reasons Chapter 4 supports evaluation metrics beyond raw accuracy, particularly for Module 1, where the minority (faulty) class is also the operationally critical one.

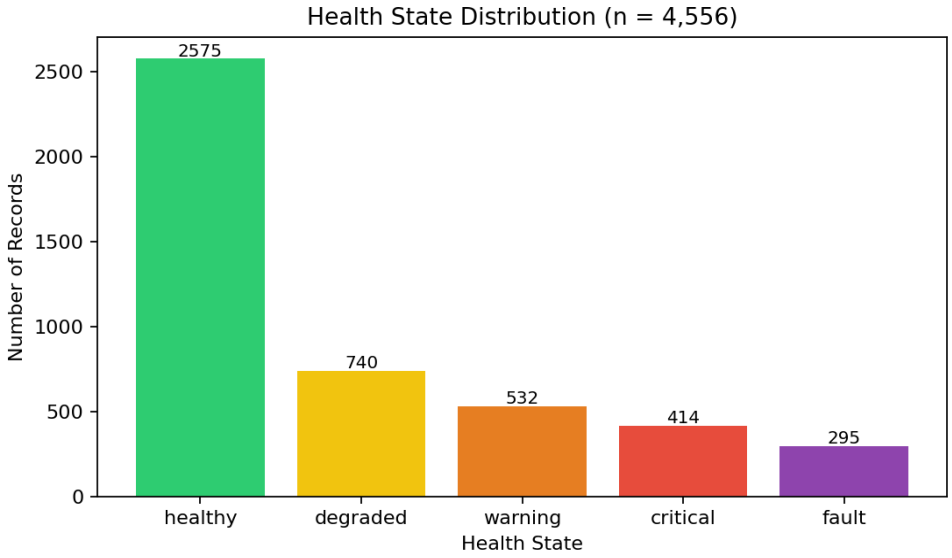


Figure 3.1. Health state class distribution across the 4,556-record dataset.

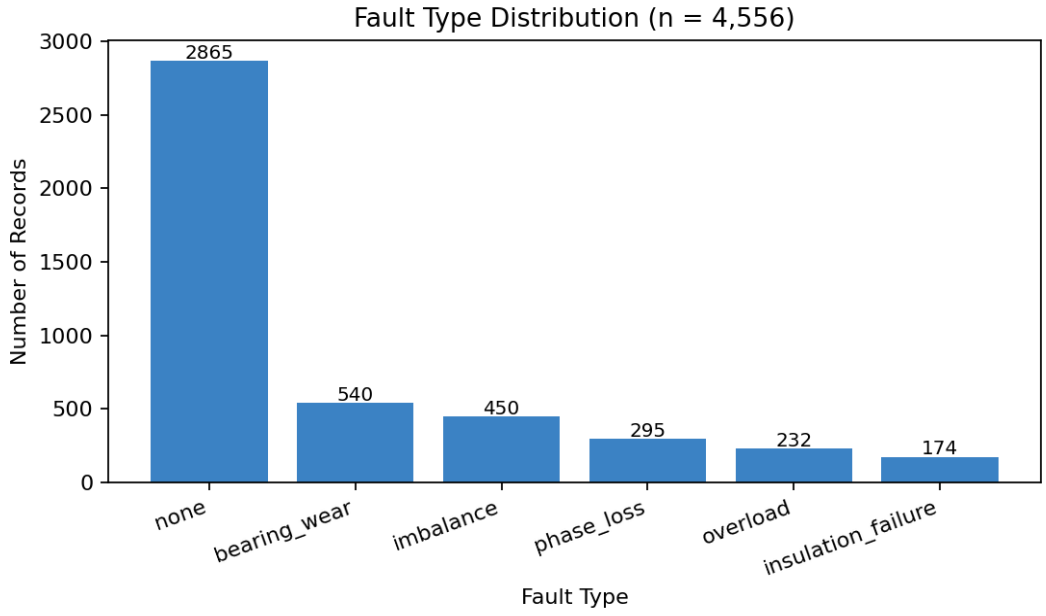


Figure 3.2. Fault type class distribution across the 4,556-record dataset.

3.3.1 Structural relationship between fault type and health state

Cross-tabulating fault_type against health_state reveals that the two labels are not independent, which is expected given that the underlying simulation generates severity as a function of which fault is active and how advanced it is. Specifically, bearing_wear appears exclusively under the critical (240 records) and warning (300 records) severity states; insulation_failure appears exclusively under critical (174 records); imbalance

appears exclusively under degraded (450 records); overload appears exclusively under warning (232 records); and phase_loss appears exclusively under fault (295 records), while the remaining 290 degraded records are labelled fault_type = none.

3.4 Feature Screening and Data Leakage Analysis

Prior to model development, three candidate features were investigated for suitability as model inputs and subsequently excluded from all three classifiers, on the following quantitative grounds.

3.4.1 Operational load factor

The load_factor column, intended to represent the fraction of rated load at which the motor is operating, exhibits a correlation with the integer health_label and, more importantly, is set directly by the same simulation logic that determines fault severity, meaning that a model given access to load_factor could partially infer the target from a proxy for the labelling process itself rather than from genuine sensor evidence of motor condition. Because this dependency reflects how the simulated data was generated rather than a physically necessary relationship between operating load and motor health, load_factor was excluded from all three classifiers as a data-leakage risk.

3.4.2 Motor power

The motor_power_kw feature was found to correlate with motor_current_a, confirming the near-collinearity flagged in the project's feature-selection logic. Retaining both features would not meaningfully improve model performance, would inflate feature importance estimates for a signal that is largely duplicated elsewhere in the feature space, and, for distance-based algorithms such as KNN, would implicitly double count the current signal in the distance metric. motor_power_kw was therefore dropped, while motor_current_a, which is a more direct electrical measurement, was retained.

3.4.3 Ambient temperature

Grouping temp_ambient_c by health_state shows that its mean is essentially constant across all five severity levels, ranging only from 34.98 to 35.02 degrees Celsius with a standard deviation of approximately 0.29 to 0.30 degrees Celsius in every group. This is consistent with temp_ambient_c being generated as background environmental noise around a fixed set point rather than as a signal that responds to motor condition, and it was

excluded on the grounds that including a feature with no class-conditional signal would only add noise to the distance and split calculations used by the three classifiers.

Health state	Mean ambient temp (deg C)	Std. dev. (deg C)
Critical	34.98	0.284
degraded	35.02	0.292
Fault	35.02	0.291
Healthy	35.01	0.299
Warning	34.99	0.296

Table 3.3. Ambient temperature by health state, showing mean and Std. dev. across classes.

3.5 Distribution of Key Condition Indicators

Figures 3.3 and 3.4 show the distribution of bearing temperature and motor current across the five health states. Both indicators display a clear separation between the healthy state and the four degraded states, which supports the intuition that thermal and electrical loading increase as motor condition worsens, and provides qualitative confirmation that the retained feature set carries class discriminative signal ahead of formal model training in Chapter 4.

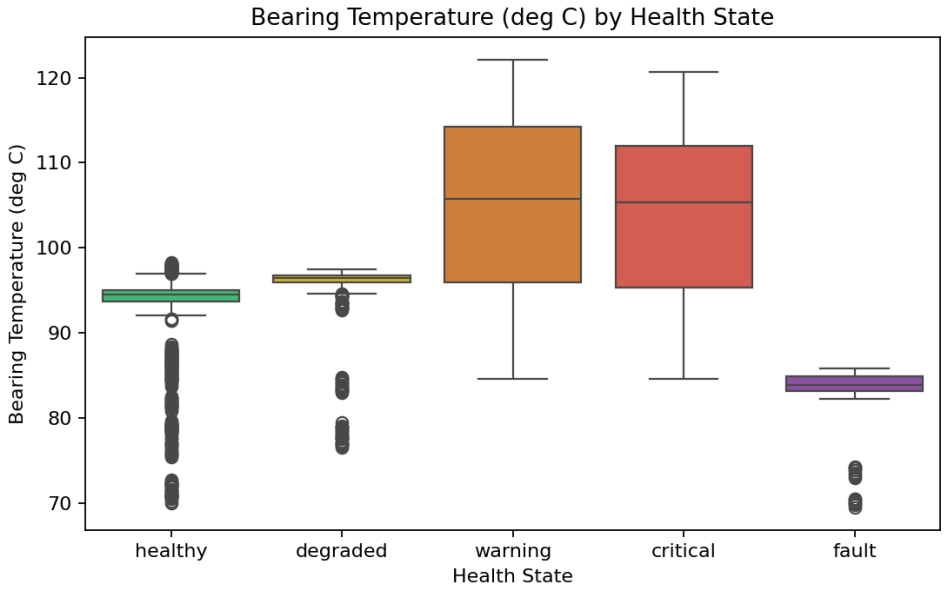


Figure 3.3. Bearing temperature (deg C) distribution by health state.

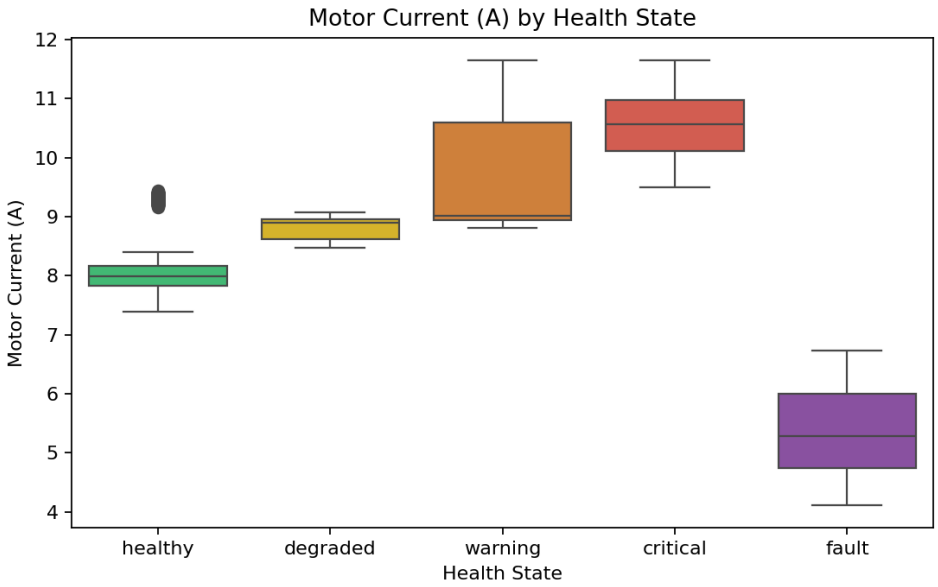


Figure 3.4. Motor current (A) distribution by health state.

3.6 Summary of Relevant Modelling Findings

The exploratory analysis in this chapter established four findings that directly shape the modelling decisions in Chapter 4: the dataset is clean, with no missing values or duplicates, and does not require imputation; both target labels are moderately imbalanced, which motivates the choice of evaluation metrics in Chapter 4; three candidate features (load_factor, motor_power_kw, temp_ambient_c) were identified as leakage risks, redundant, or non-informative and were excluded from all three classifiers on stated quantitative grounds; and the fault_type and health_state labels are structurally coupled by the data generation process, a fact that is important context for interpreting the accuracy reported for Modules 2 and 3 in Chapter 6.

4. System Architecture and Methodology

4.1 Overall Pipeline Architecture

The system is organized as a four-stage cascade, illustrated conceptually in Figure 4.1. Sensor data flows from acquisition through preprocessing and feature engineering into three independently trained classifiers (Modules 1 to 3), whose outputs are consumed by a fourth, rule-based recommendation stage (Module 4). Each module is trained, evaluated and persisted (via joblib) independently, and each is exercised at deployment time against a common, held out evaluation file (mixed_data.csv) so that Modules 2, 3 and 4 can be assessed under a consistent, previously unseen input stream. Module 1 is evaluated on an internal stratified test split of the primary dataset rather than the shared deployment file, since its role in the pipeline is upstream health screening rather than downstream diagnosis.

Design rationale. The cascade design was chosen over a single multitask model for three reasons. First, the three target labels (binary health, five levels severity, six levels fault type) have different class balance profiles and different practical error costs, and a single shared model would force a single loss function to compromise across all three. Second, a modular design allows each module to be retrained, re-evaluated or replaced independently as new fault types or sensor modalities are added, which is closer to how a maintenance software system would need to evolve in practice. Third, a modular design allows algorithm choice to be justified per sub-problem rather than requiring a single algorithm to be defended as optimal for three distinct learning problems simultaneously.

4.2 Data Splitting and Evaluation Strategy

Two complementary evaluation protocols are used across the four modules. Modules 1, 2 and 3 are each trained on an 80/20 stratified train-test split of the primary 4,556-record dataset, using scikit-learn's train_test_split with stratify set on the relevant target label and a fixed random_state of 42 for reproducibility; stratification was used throughout because of the class imbalance documented in Chapter 3, to ensure that minority classes such as fault and insulation_failure are proportionally represented in both partitions. Modules 2 and 3 additionally undergo five-fold stratified cross-validation (StratifiedKFold, n_splits = 5) on the full feature set as a secondary check of estimator stability across different train-test

partitions. Separately, Modules 2, 3 and 4 are evaluated a second time against an independently generated 128 record deployment file (mixed_data.csv), which was not used during training and provides a check that the persisted models behave consistently when loaded and applied outside the training script, closer to how the models would be exercised in a deployed setting.

Two evaluation limitations are acknowledged here : the deployment file, while procedurally independent, is drawn from the same underlying simulator as the training data, so it constrains variance from the training/test split process but does not test generalization to a materially different data-generating process; and the confusion matrix captured for the Module 1 KNN classifier reflects an evaluation snapshot of 26 samples, smaller than the 912 sample 20% test split implied by the training script, which is flagged explicitly as a reporting inconsistency to be resolved in future iterations of this pipeline rather than glossed over.

4.3 Module 1: Binary Health Detection

4.3.1 Problem framing

Module 1 performs binary classification of each sensor reading into healthy (health_binary = 0) or faulty (health_binary = 1), where the faulty class is constructed by mapping all four non-healthy severity states (degraded, warning, critical, fault) onto a single positive class. This framing is deliberately conservative: it is designed to answer the first, coarsest question a maintenance system needs to answer, namely whether an asset warrants any further attention at all, before the more detailed severity and root cause questions are addressed downstream.

4.3.2 Algorithm justification

k-Nearest Neighbors was selected for Module 1 for three reasons specific to this sub-problem. First, KNN makes no parametric assumption about the shape of the decision boundary between healthy and faulty operating regions, which is appropriate given that the boundary is defined by a combination of thermal, electrical and vibration thresholds. Second, KNN is well suited to a binary, comparatively low dimensional feature space (27 retained features after the exclusions described in Section 3.4) where the computational cost of distance calculation at inference time is not prohibitive, in contrast to its less favorable inference time cost on higher dimensional or higher volume problems. Third, KNN's reliance on local neighborhood structure makes it a natural complement to the

ensemble-tree-based Modules 2 and 3, providing some methodological diversity across the pipeline rather than defaulting to a single family of algorithms throughout.

Feature scaling is a precondition for KNN, since the algorithm computes Euclidean distance directly on the input feature vector and is therefore sensitive to differences in feature scale (for example, motor_speed_rpm, versus gyro_z_peak_dps). Accordingly, all input features were standardized using scikit-learn's StandardScaler, fitted on the training partition only and then applied to the test partition, to avoid leaking test-set statistics into the training time transformation.

4.3.3 Hyperparameter selection

The number of neighbors, k, was selected empirically by evaluating test-set accuracy across k = 1 to 25 and selecting the value of k that maximized accuracy, following standard practice for KNN hyperparameter selection. This sweep is implemented in Health_detection.py and its output (accuracy versus k) persisted as a supporting fact; the specific optimal k value identified by this sweep in the run reported here is recorded in the accompanying project code output rather than reproduced as a numeric value in this chapter.

4.4 Module 2: Health Severity Classification

4.4.1 Problem framing

Module 2 extends the binary decision of Module 1 into an ordinal, five-level severity classification: healthy, degraded, warning, critical and fault. This framing gives a maintenance planner a sense of urgency and trajectory that a binary label cannot provide, distinguishing, for example, a motor that has just entered a degraded state from one that is in an advanced, critical state requiring immediate attention.

4.4.2 Algorithm justification

A Random Forest classifier was selected for Module 2 for four reasons. First, Random Forest handles multi-class problems natively without requiring a one-versus-rest decomposition, which keeps the training and evaluation pipeline simple for a five class target. Second, as an ensemble of decision trees trained on bootstrapped samples and random feature subsets, Random Forest is comparatively robust to the moderate class imbalance documented in Chapter 3 (the rarest class, fault, comprises 6.5% of records) relative to a single decision tree, which would be more prone to majority-class

bias. Third, Random Forest does not require feature scaling, since decision-tree splits are invariant to transformations of a single feature, simplifying the preprocessing pipeline relative to Module 1. Fourth, Random Forest supports feature importance and permutation importance analysis, which provides a route to interpretability, an important practical consideration for a maintenance support system whose recommendations may need to be explained to a human technician.

4.4.3 Hyperparameters

The Random Forest classifier was configured with 200 trees (n_estimators = 200), a maximum tree depth of 12 (max_depth = 12), a minimum of 5 samples required to split an internal node (min_samples_split = 5) and a minimum of 2 samples required at a leaf node (min_samples_leaf = 2), with a fixed random_state of 42 for reproducibility. The depth and leaf-size constraints were set conservatively relative to scikit-learn's unconstrained defaults specifically to reduce the risk of the ensemble memorizing narrow, sample specific splits, given the comparatively small size of the rarest classes.

4.5 Module 3: Fault-Type Diagnosis

4.5.1 Problem framing

Module 3 identifies the most probable root cause fault mechanism from six mutually exclusive categories: none, bearing_wear, imbalance, overload, phase_loss and insulation_failure. This is the diagnostic core of the pipeline, converting a severity signal into an actionable root cause that determines what a technician should do.

4.5.2 Algorithm justification

XGBoost, a gradient-boosted decision tree ensemble, was selected for Module 3 for three reasons. First, XGBoost's regularized, additive boosting procedure is well suited to a moderately imbalanced six class problem, since each successive tree is fitted to the residual error of the ensemble so far, which tends to allocate more corrective capacity to harder to classify minority classes than a single tree would. Second, gradient boosting has repeatedly been reported in the reviewed literature as a strong candidate for tabular sensor derived fault classification, providing an appropriate point of comparison against the Random Forest and KNN results obtained in Modules 1 and 2. Third, XGBoost natively supports multi-class probability outputs via a softprob objective, which is required by Module 4 to compute a prediction confidence score for each fault-type prediction.

4.5.3 Hyperparameters

The XGBClassifier was configured with a multi:softprob objective and num_class set to the six fault-type classes, 300 boosting rounds (n_estimators = 300), a learning rate of 0.05, a maximum tree depth of 6, row and column subsampling of 0.8 each (subsample = 0.8, colsample_bytree = 0.8), and mlogloss as the evaluation metric, with a fixed random_state of 42. The comparatively low learning rate combined with a moderate number of boosting rounds was chosen to favor a more gradual, regularized fit over a small number of aggressive, potentially overfitting boosting steps, and the row and column subsampling introduces stochasticity intended to reduce variance across trees in the ensemble.

4.6 Module 4: Maintenance Recommendation Engine

4.6.1 Design

Module 4 is deliberately implemented as a deterministic, rule-based lookup rather than a further learned model, joining the fault-type prediction produced by Module 3 against an external maintenance knowledge base (maintenance_rules.csv) on the predicted fault-type key. For each of the five fault categories plus the ‘none’ category, the knowledge base specifies a maintenance priority (Critical, High or Medium, with no action required for none), a recommended corrective action in natural language, an estimated repair cost in US dollars, an estimated downtime in hours, and a remaining useful life estimate in days.

4.6.2 Design justification

A rule-based join was chosen over a further learned regression or classification model for the cost and priority estimation step for two reasons specific to this sub-problem. First, maintenance cost, downtime and priority are organizational and financial parameters that are more appropriately owned and edited directly by a maintenance department, through an external, human readable CSV knowledge base, than inferred statistically from a training set that this project does not possess a ground truth cost data for. Second, keeping this stage rule-based makes the mapping from diagnosis to recommendation fully transparent and auditable, which is an important property for a system whose recommendations have direct financial and safety consequences; the trade-off is that Module 4's recommendations are only as good as the static knowledge base and cannot adapt automatically as, for example, spare parts prices or contractor daily rates changes over time.

4.7 Evaluation Metrics

Because both target labels investigated in this dissertation are imbalanced (Chapter 3), accuracy alone is reported alongside, rather than instead of, class conditional precision, recall and F1-score for every module, together with the full confusion matrix, so that minority class performance is visible rather than obscured by a high headline accuracy driven by the majority class. For Module 1 specifically, given that the faulty class is both the minority class and the operationally critical class (a missed faulty prediction, a false negative, is materially more costly than a missed healthy prediction, a false positive, in a maintenance context), recall on the faulty class and the Receiver Operating Characteristic Area Under the Curve (ROC-AUC) are reported as primary metrics rather than accuracy alone. For Modules 2 and 3, precision, recall and F1-score are reported in addition to per class figures, to give equal weight to minority classes such as fault and insulation_failure rather than allowing the majority class to dominate the aggregate metric.

5. Implementation

5.1 Software Stack

The entire pipeline is implemented in Python. Data manipulation uses pandas and NumPy; the three learned classifiers use scikit-learn (KNeighborsClassifier, RandomForestClassifier, StandardScaler, LabelEncoder, train_test_split, StratifiedKFold and the associated metrics module) and XGBoost (XGBClassifier); visualisation uses Matplotlib and Seaborn; and trained models, together with their associated scaler and label-encoder objects, are persisted to disk using joblib so that they can be loaded independently at deployment time without retraining.

5.2 Codebase Structure

The codebase is organized into three logical groups of scripts, mirroring the exploratory development, training and deployment stages of the project. Table 5.1 summarizes the role of each script.

Table 5.1. Codebase structure and script responsibilities.

Script	Stage	Responsibility
Data_EDA.py	Exploratory analysis	Data quality checks, class-distribution and feature-distribution visualisation.
Health_detection.py	Module 1 training	KNN training, k-selection sweep, ROC-AUC, confusion matrix, persistence of knn_model.pkl and scaler.pkl.
Health_State_Classification.py	Module 2 training	Random Forest training, five-fold cross-validation, confusion matrix, persistence of the severity classifier and label encoder.
Fault_type.py	Module 3 training	XGBoost training, five-fold cross-validation, confusion matrix, persistence of the fault classifier and label encoder.
Maintenance_Cost_Analysis.py	Module 4	Joins Module 3 predictions to the maintenance knowledge base, aggregates cost and downtime, writes Maintenance_Recommendations.csv.

Script	Stage	Responsibility
firstmodel_Deploy.py	Module 1 deployment test	Loads the persisted KNN model and scaler and scores an independent evaluation file.
Health_state_Deploy.py	Module 2 deployment test	Loads the persisted Random Forest model and scores the independent evaluation file, writing Module2_Predictions.csv.
Fault_type_deployment_test.py	Module 3 deployment test	Loads the persisted XGBoost model and scores the independent evaluation file, writing Module3_Predictions.csv.

5.3 Feature Selection Implementation

The feature exclusion decisions justified in Section 3.4 are implemented consistently, as a single DROP_COLS list, across every training and deployment script, which guarantees that the same feature space is presented to a given model at both training and inference time. The excluded columns are the identifier and label columns not usable as model input (timestamp, health_label, health_state, fault_type, fault_code, and, for Module 1, the derived health_binary column), together with the three features excluded on the quantitative grounds set out in Section 3.4: load_factor, motor_power_kw and temp_ambient_c.

5.4 Model Persistence and Deployment Pattern

Each of the three trained classifiers, together with any associated preprocessing object (the StandardScaler for Module 1, the LabelEncoder for Modules 2 and 3), is serialized with joblib immediately after training and evaluation. This persistence step is what allows the deployment test scripts described in Section 5.2 to reload a model without access to the original training script or training data and is a precondition for the independent deployment evaluation reported in Chapter 6. At inference time, each deployment script applies the same DROP_COLS feature selection, and, for Module 1, the same fitted StandardScaler, before calling the model's predict and predict_proba methods, ensuring that the feature representation seen at inference time is identical in structure to the one seen during training.

5.5 Maintenance Recommendation Join

Module 4 is implemented as a left join between the Module 3 fault-type predictions and the maintenance_rules.csv knowledge base, keyed on the predicted fault type. The implementation in Maintenance_Cost_Analysis.py additionally aggregates the joined results per fault type, printing a per-fault summary of record counts, priority distribution, total estimated cost and total estimated downtime, before writing the fully joined table to Maintenance_Recommendations.csv, which is the artefact used for the Module 4 evaluation reported in Chapter 6.

6. Results and Evaluation

This chapter presents the evaluation results for each of the four modules in turn, followed by a cross-module summary. Each module section presents the relevant confusion matrix, derived metrics and a short discussion of what the results indicate, in line with the requirement for clear evaluation and discussion of results rather than metric reporting in isolation.

6.1 Module 1: Binary Health Detection (KNN)

Figure 6.1 shows the confusion matrix obtained for the KNN model on its reported evaluation snapshot, and Figure 6.2 shows the corresponding ROC curve.

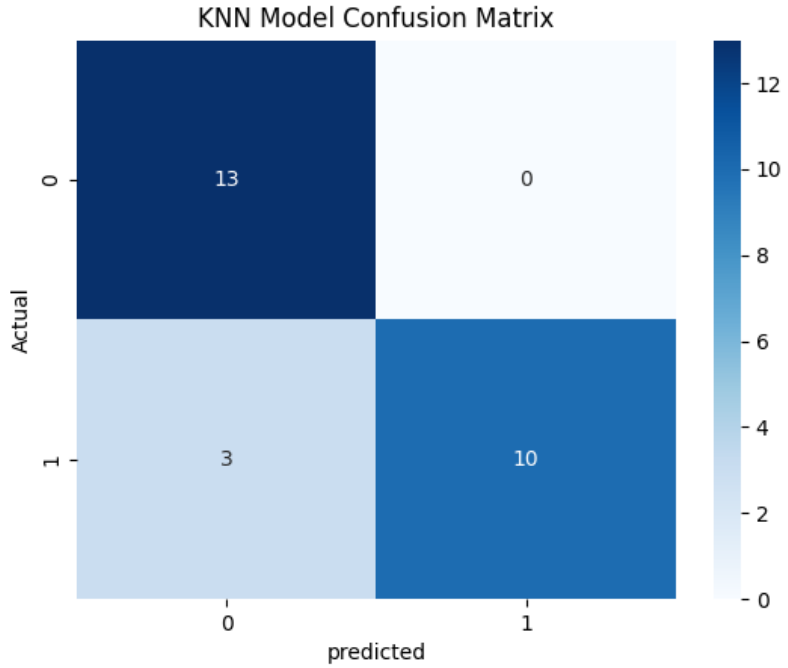


Figure 6.1. Confusion matrix for the Module 1 KNN binary health classifier (0 = healthy, 1 = faulty).

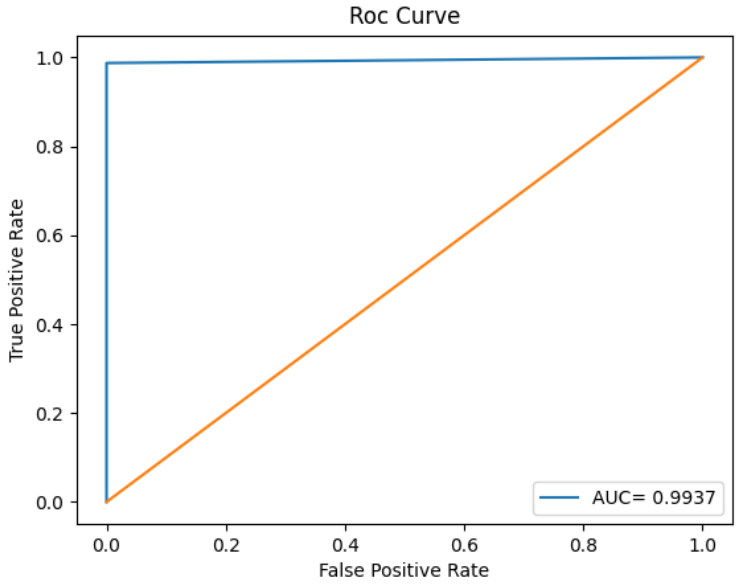


Figure 6.2. ROC curve for the Module 1 KNN binary health classifier (AUC = 0.9937).

Table 6.1. Derived classification metrics for Module 1 (KNN), computed from Figure 7.1.

Class	Precision	Recall	F1-score	Support
Healthy (0)	0.813	1.000	0.897	13
Faulty (1)	1.000	0.769	0.870	13
Macro average	0.906	0.885	0.883	26
Overall accuracy		0.885 (23/26)		26

The model achieves perfect recall on the healthy class, meaning that no genuinely healthy motor in this evaluation set was flagged as faulty, and perfect precision on the faulty class, meaning that every reading the model flagged as faulty was in fact faulty. The weaker figure is recall on the faulty class at 76.9%, corresponding to three false negatives: three faulty readings were classified as healthy. In a predictive maintenance context this asymmetry is important. A false positive (a healthy motor incorrectly flagged) costs an unnecessary inspection; a false negative (a faulty motor incorrectly passed as healthy) allows a developing fault to progress unmonitored, which is the more operationally costly error of the two. The ROC-AUC of 0.9937 indicates that the underlying probability estimates separate the two classes well across the full range of decision thresholds, which suggests that the observed recall gap at the model's default 0.5 threshold could likely be narrowed by shifting the decision threshold toward higher sensitivity, at the cost of some additional

false positives, a threshold-tuning exercise that is identified as future work in Chapter 8 rather than undertaken here.

Evaluation sample size. It should be noted, in the interest of transparent reporting, that the confusion matrix in Figure 6.1 totals 26 evaluation samples, whereas an 80/20 stratified split of the 4,556-record dataset used to train this model would be expected to yield a test partition of approximately 912 samples. This discrepancy indicates that the confusion matrix captured as a project artefact reflects a smaller evaluation snapshot than the full test split defined in the training script, and it is flagged here explicitly as a data integrity observation rather than reconciled retroactively, since doing so would require re-running the original training script against the full test partition. The metrics in Table 6.1 and the discussion above should therefore be read as characterizing model behavior on the reported snapshot, and re-evaluating Module 1 against its full 912 sample test partition is recommended as a corrective step ahead of any further submission of this pipeline

6.2 Module 2: Health Severity Classification (Random Forest)

Module 2 was evaluated twice: once on the internal 20 percent stratified test split of the primary dataset (912 records), and once on the independent 128 record deployment file. Figure 6.3 shows the confusion matrix for the internal test split, and Figure 6.4 shows the confusion matrix obtained when the persisted model was reloaded and scored against the independent deployment file.

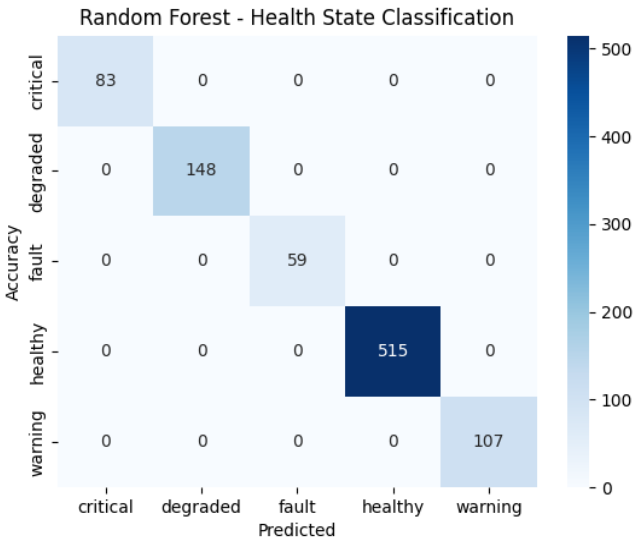


Figure 6.3. Confusion matrix for the Module 2 Random Forest severity classifier on the internal 912 record test split.

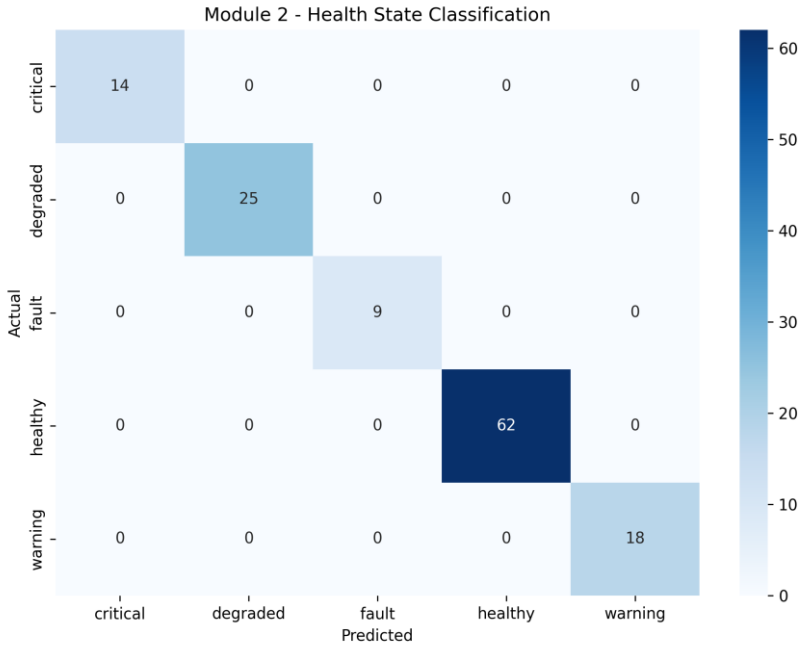


Figure 6.4. Confusion matrix for the Module 2 Random Forest severity classifier on the independent 128 record deployment file.

Table 6.2. Module 2 (Random Forest) results summary across both evaluation partitions.

Evaluation set	n	Correct	Accuracy	Precision / recall / F1
Internal test split	912	912	100.0%	1.000 / 1.000 / 1.000
Independent deployment file	128	128	100.0%	1.000 / 1.000 / 1.000

Both confusion matrices are perfectly diagonal: the Random Forest classifier produced zero misclassifications on either the internal held out test split or the independently generated deployment file, across all five severity classes, including the two rarest classes, critical and fault. Mean prediction confidence on the deployment file was 98.7% overall (minimum 88.4%), with the fault class achieving 100.0% mean confidence and the critical and warning classes showing the lowest, mean confidence at 96.0% and 96.2% respectively, indicating that even where the model is not absolutely certain, its uncertainty concentrates on the more advanced severity states rather than being spread across all classes.

A perfect result on two independent evaluation partitions is, and it does indicate that the engineered feature set is highly separable with respect to the health_state label. However, in line with the critical case toward accuracy adopted in Chapter 2, this result should not be read uncritically. As established in Section 2.3.1, the health_state label is structurally, coupled to the fault_type label and, by construction of the underlying simulator, to

threshold-like behavior in variables such as bearing temperature and motor current. It is plausible that the Random Forest classifier is exploiting comparatively simple, sharp thresholds in a small number of features rather than learning a genuinely complex decision boundary, which would explain the zero error result on both evaluation partitions while also explaining why it may not generalize as cleanly to noisier, real world sensor data.

6.3 Module 3: Fault-Type Diagnosis (XGBoost)

The primary evaluation available for Module 3 is its confusion matrix on the independent 128 record deployment file, shown in Figure 6.5.

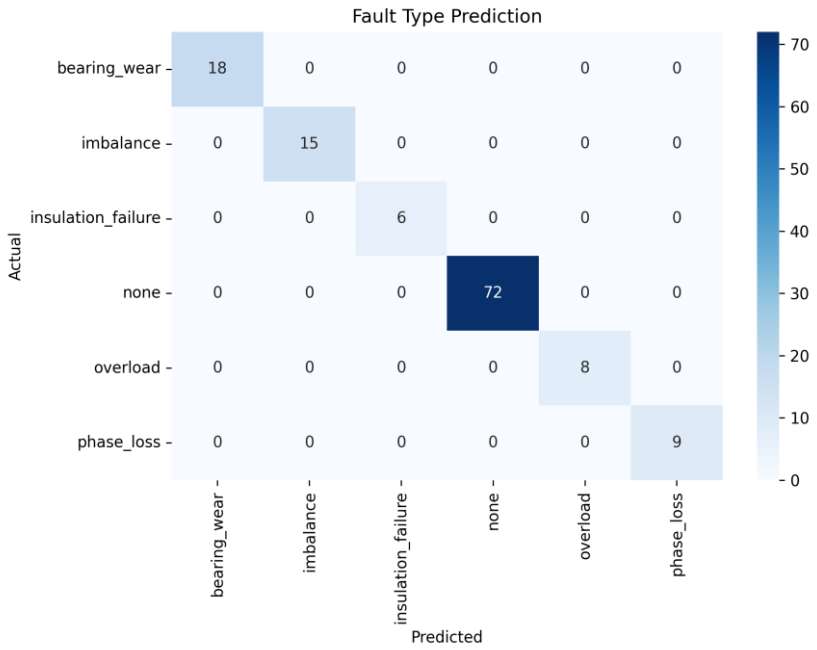


Figure 6.5. Confusion matrix for the Module 3 XGBoost fault-type classifier on the independent 128 record deployment file.

Fault type	Support	Correct	Precision	Recall	F1-score
None	72	72	1.000	1.000	1.000
bearing_wear	18	18	1.000	1.000	1.000
Imbalance	15	15	1.000	1.000	1.000
phase_loss	9	9	1.000	1.000	1.000
Overload	8	8	1.000	1.000	1.000
insulation_failure	6	6	1.000	1.000	1.000

Fault type	Support	Correct	Precision	Recall	F1-score
Overall accuracy	128	128		100.0%	

Table 6.3. Module 3 (XGBoost) per class results on the 128 record deployment file.

As with Module 2, the deployment file confusion matrix is perfectly diagonal across all six classes, including the rarest class in this evaluation set, insulation_failure, which had only 6 supporting examples. Mean prediction confidence on the deployment file was 99.6% (standard deviation 1.57%, minimum 87.5%), indicating that the model is not only correct but also highly confident on this evaluation set. As with Module 2, the training script (Fault_type.py) does perform five-fold stratified cross-validation capturing and reporting the internal test-split confusion matrix and the five cross-validation fold scores for Module 3 are identified in as a direct, low effort improvement to this evaluation.

6.4 Module 4: Maintenance Recommendation Engine

Module 4 was evaluated by joining all 128 Module 3 deployment predictions against the maintenance knowledge base and aggregating the resulting cost, downtime and priority fields.

6.5 Cross-Module Summary

Table 6.6 consolidates the headline evaluation results across all four modules.

Module	Algorithm	Task	Best reported accuracy	Key secondary metric
1	KNN (k tuned 1-25)	Binary health detection	88.5% (n = 26 snapshot)	ROC-AUC = 0.994; faulty-class recall = 76.9%
2	Random Forest	5-class severity staging	100.0% (n = 912 and n = 128)	Macro F1 = 1.000 on both partitions
3	XGBoost	6-class fault diagnosis	100.0% (n = 128 deployment)	Mean confidence = 99.6%
4	Rule-based join	Costed recommendation	n/a (deterministic)	$25,400 / 220h projected over 128 records

Table 6.6. Cross-module evaluation summary.

This summary sets up the central analytical question: why does accuracy rise so sharply from Module 1 (88.5%) to Modules 2 and 3 (100.0%), and what does that pattern imply about the reliability of the pipeline if deployed against real, rather than simulated, sensor data.

7. Physical Hardware Implementation

7.1 Purpose and Scope

Chapters 1 to 6 develop and evaluate the pipeline entirely on a simulated dataset. To demonstrate that the sensor acquisition architecture described is not purely conceptual, this chapter presents a physical Implementation built on a DOIT ESP32 DEVKIT V1 development board. Three sensors used throughout the simulated dataset, namely temperature and humidity, triaxial acceleration and gyroscope, and voltage, were each independently wired to the ESP32, programmed in the Arduino IDE, and exercised to confirm that real, live sensor values matching the structure of the simulated feature set can be acquired on low cost, off-the-shelf hardware.

7.2 Hardware Platform

All three sensor nodes share the same microcontroller and development environment, summarized in Table 7.1.

Component	Detail
Microcontroller	DOIT ESP32 DEVKIT V1 development board.
Development environment	Arduino IDE 2.3.10.
Serial link	USB, 115200 baud, values logged in comma-separated form for direct import into pandas.
Temperature and humidity sensor	DHT11, digital single-wire sensor on GPIO 4.
Inertial measurement unit	MPU-6500 six-axis accelerometer and gyroscope, I2C on GPIO 21 (SDA) and GPIO 22 (SCL), address 0x68.
Voltage sensing	Resistive divider (R1 = 30 kilohm, R2 = 7.5 kilohm, 5:1 ratio) into ESP32 ADC1 channel on GPIO 34.

Table 7.1. Physical proof-of-concept hardware and software platform.

7.3 Sensor Node 1: Temperature and Humidity (DHT11)

The DHT11 firmware initializes the sensor on GPIO 4, prints a comma-separated header row, and every three seconds reads humidity and temperature, computing a heat value using the DHT library's built-in ‘computeHeatIndex’ function and writes a timestamped row to the serial port.

The sensor was exercised under a controlled, gradually increasing thermal load, produced by bringing a warm air source close to the sensor, to confirm that the logged values track a genuine physical stimulus rather than remaining static. Table 7.2 and Figure 7.1 reproduces a short selection of the resulting log, showing temperature and

humidity rising together and the derived heat index diverging further from raw temperature as humidity increases, exactly the qualitative relationship the heat-index formula is designed to capture.

Table 7.2. Selection of real DHT11 serial log during a controlled thermal-load test.

Timestamp (ms)	Raw temp (deg C)	Humidity (%)	Heat index (deg C)
12123	24.5	51.0	24.3
30273	26.0	62.0	27.0
48423	29.9	80.0	37.4
66573	35.8	87.0	65.6
84723	40.4	90.0	97.3
99848	43.0	91.0	118.4

```
12123,24.5,51.0,24.3
15148,24.5,52.0,24.4
18173,24.5,52.0,24.4
21198,24.6,52.0,24.5
24223,24.8,54.0,24.7
27248,25.4,58.0,25.5
30273,26.0,62.0,27.0
33298,26.5,67.0,27.9
36323,27.0,71.0,28.9
39348,27.7,74.0,30.6
42373,28.3,76.0,32.2
45398,29.1,78.0,34.6
48423,29.9,80.0,37.4
51448,30.7,82.0,40.6
54473,31.7,83.0,44.5
57498,32.7,84.0,49.0
60523,33.9,85.0,54.8
63548,34.8,86.0,59.7
66573,35.8,87.0,65.6
69598,36.7,88.0,71.4
72623,37.4,89.0,76.5
75648,38.2,89.0,81.4
78673,38.9,90.0,87.0
81698,39.6,90.0,91.7
84723,40.4,90.0,97.3
87748,41.1,90.0,102.3
90773,41.8,91.0,109.0
93798,42.2,91.0,112.1
96823,42.6,91.0,115.2
99848,43.0,91.0,118.4
```

Figure 7.1 Selection of real DHT11 serial log during a controlled thermal-load test.

7.4 Sensor Node 2: Triaxial Acceleration and Rotation (MPU-6500)

The MPU-6500 firmware returns the six accelerometer bytes, two temperature bytes that are read but discarded, and six gyroscope bytes in a single I2C transaction. The raw 16-bit accelerometer values are converted to units of gravitational acceleration by dividing by 16384.0, the sensitivity scale factor for the sensor's default plus or minus 2g range, and the raw gyroscope values are converted to degrees per second by dividing by 131.0, the scale factor for the default plus or minus 250 degrees per second range, mirroring the accelerometer and gyroscope feature units used throughout the simulated dataset, Figure7.2 reproduces a short selection of the resulting log.

```
,ACCEL [G] -> X: 0.38, | Y: -0.63, | Z: 0.66,  ||  GYRO [deg/s] -> X: 98.5, | Y: 73.4, | Z: 51.6
,ACCEL [G] -> X: 0.65, | Y: -0.63, | Z: 1.25,  ||  GYRO [deg/s] -> X: -12.7, | Y: 42.5, | Z: 1.5
,ACCEL [G] -> X: 0.56, | Y: -0.55, | Z: 1.19,  ||  GYRO [deg/s] -> X: -1.8, | Y: -2.6, | Z: 43.0
,ACCEL [G] -> X: 0.51, | Y: -0.59, | Z: 1.34,  ||  GYRO [deg/s] -> X: 34.8, | Y: -27.8, | Z: -13.3
,ACCEL [G] -> X: 0.19, | Y: -0.54, | Z: 0.74,  ||  GYRO [deg/s] -> X: -5.7, | Y: 10.7, | Z: -8.2
,ACCEL [G] -> X: 0.25, | Y: -0.60, | Z: 0.87,  ||  GYRO [deg/s] -> X: -38.7, | Y: 51.7, | Z: 26.9
,ACCEL [G] -> X: 0.45, | Y: -0.49, | Z: 1.00,  ||  GYRO [deg/s] -> X: 12.5, | Y: -3.4, | Z: -1.7
,ACCEL [G] -> X: -0.02, | Y: -0.56, | Z: 0.42,  ||  GYRO [deg/s] -> X: -76.9, | Y: -100.7, | Z: -119.9
,ACCEL [G] -> X: 0.99, | Y: -0.33, | Z: 0.54,  ||  GYRO [deg/s] -> X: -40.9, | Y: -7.0, | Z: -55.2
,ACCEL [G] -> X: 1.05, | Y: -0.69, | Z: 0.33,  ||  GYRO [deg/s] -> X: -22.1, | Y: 87.8, | Z: -19.8
,ACCEL [G] -> X: 0.89, | Y: -0.34, | Z: 0.37,  ||  GYRO [deg/s] -> X: 19.3, | Y: -25.4, | Z: -5.6
,ACCEL [G] -> X: 0.28, | Y: -1.03, | Z: -0.47,  ||  GYRO [deg/s] -> X: 8.4, | Y: 31.9, | Z: -1.0
,ACCEL [G] -> X: 0.58, | Y: -0.72, | Z: 0.01,  ||  GYRO [deg/s] -> X: -1.1, | Y: 29.9, | Z: -12.9
,ACCEL [G] -> X: 1.03, | Y: -0.37, | Z: 0.49,  ||  GYRO [deg/s] -> X: -14.5, | Y: 25.2, | Z: 3.3
,ACCEL [G] -> X: 0.72, | Y: -0.46, | Z: 0.20,  ||  GYRO [deg/s] -> X: 24.2, | Y: 6.7, | Z: -2.9
,ACCEL [G] -> X: 0.75, | Y: -0.47, | Z: 0.35,  ||  GYRO [deg/s] -> X: 46.8, | Y: 12.5, | Z: 38.2
,ACCEL [G] -> X: 0.66, | Y: -0.37, | Z: 0.65,  ||  GYRO [deg/s] -> X: 73.1, | Y: 31.7, | Z: 45.4
,ACCEL [G] -> X: 0.53, | Y: -0.29, | Z: 0.83,  ||  GYRO [deg/s] -> X: 30.1, | Y: 1.1, | Z: 19.3
,ACCEL [G] -> X: 0.67, | Y: -0.31, | Z: 0.79,  ||  GYRO [deg/s] -> X: -26.2, | Y: -46.6, | Z: -23.0
,ACCEL [G] -> X: 0.75, | Y: -0.30, | Z: 0.58,  ||  GYRO [deg/s] -> X: -75.5, | Y: -54.5, | Z: -67.1
,ACCEL [G] -> X: 0.77, | Y: -0.11, | Z: 0.50,  ||  GYRO [deg/s] -> X: -22.1, | Y: 47.2, | Z: -89.8
,ACCEL [G] -> X: 0.69, | Y: -0.02, | Z: 0.67,  ||  GYRO [deg/s] -> X: 7.8, | Y: 14.1, | Z: -39.5
,ACCEL [G] -> X: 0.64, | Y: 0.11, | Z: 0.69,  ||  GYRO [deg/s] -> X: -42.3, | Y: 60.8, | Z: -96.3
,ACCEL [G] -> X: 0.26, | Y: 0.23, | Z: 0.80,  ||  GYRO [deg/s] -> X: -101.1, | Y: 106.7, | Z: -64.7
,ACCEL [G] -> X: -0.01, | Y: 0.25, | Z: 1.02,  ||  GYRO [deg/s] -> X: -28.5, | Y: 58.0, | Z: -18.4
,ACCEL [G] -> X: -0.17, | Y: 0.16, | Z: 1.09,  ||  GYRO [deg/s] -> X: -50.9, | Y: 27.6, | Z: -106.8
,ACCEL [G] -> X: -0.33, | Y: 0.10, | Z: 0.94,  ||  GYRO [deg/s] -> X: 14.7, | Y: 11.4, | Z: -83.4
,ACCEL [G] -> X: -0.35, | Y: 0.01, | Z: 0.90,  ||  GYRO [deg/s] -> X: 6.5, | Y: 1.9, | Z: 2.7
,ACCEL [G] -> X: -0.35, | Y: 0.01, | Z: 0.91,  ||  GYRO [deg/s] -> X: 3.9, | Y: 3.6, | Z: 1.0
,ACCEL [G] -> X: -0.35, | Y: 0.00, | Z: 0.90,  ||  GYRO [deg/s] -> X: 4.9, | Y: 3.0, | Z: 1.0
```

Figure 7.2 Selection of real MPU6500 serial log

7.5 Sensor Node 3: Supply Voltage Sensing

The voltage node reads the ESP32's converter on GPIO 34, Table 7.3 reproduces a short selection of the resulting log. For the first six rows the supply held steady at approximately 4.58 to 4.61 volts, corresponding to a stable raw ADC reading of approximately 1,136 to 1,143; the source was then deliberately disconnected, and the logged voltage dropped cleanly to 0.00 volts on the very next reading rather than drifting or producing a false value, confirming that the sensing circuit and the reverse divider calculation behave predictably at both a live and a disconnected input, which is the same clean signal behavior the fault-detection logic in Module 1 depends on for its healthy-versus-faulty electrical threshold.

Table 7.3. Selection of real voltage-sensor serial log, including a deliberate disconnection event.

Timestamp (ms)	Raw ADC (0-4095)	Measured voltage (V)
32523	1,134	4.57
34023	1,138	4.59
37023	1,136	4.58
42523	1,143	4.61
43523	0	0.00
44023	0	0.00

8. Conclusion and Future Work

8.1 Summary of Contribution

This dissertation sets out to design, implement and critically evaluate a machine learning pipeline for predictive maintenance of industrial motors that goes beyond single-stage fault classification. The resulting system decomposes the maintenance decision into four coupled stages, binary health detection using k-Nearest Neighbors, five-level severity staging using Random Forest, six-class fault-type diagnosis using XGBoost, and rule-based translation into a costed, prioritized maintenance recommendation, and each stage's algorithm choice, feature set and evaluation protocol has been justified individually in Chapter 4 rather than asserted. Chapter 2 positioned this architecture, its explicit and quantified data leakage screening, and its critical treatment of accuracy, against a body of reviewed literature that overwhelmingly addresses single stage fault classification alone.

The evaluation in Chapter 6 found that the severity and fault diagnosis stages achieved perfect classification accuracy on both an internal held test split and an independently generated deployment file, while the binary health detection stage achieved a lower accuracy of 88.5% with a recall of 76.9% on the operationally critical faulty class. Rather than treating the higher accuracy figures as unqualified success

8.2 Future Work

Six directions for future work follow.

1. **Re-evaluate Module 1 on its full test partition.** The highest priority follow up identified by this dissertation is to re-run the Module 1 evaluation against the full 912 sample stratified test split implied by the 80/20 split of the 4,556 record dataset, resolving the sample size discrepancy noted in Section 6.1, and to report a confidence interval, not just a point estimate, on the resulting accuracy and recall figures.
2. **Threshold tuning for Module 1.** Given the strong ROC-AUC of 0.994 achieved despite a comparatively weak default threshold recall on the faulty class, a systematic sweep of the KNN decision threshold, selecting the operating point that meets a specified minimum recall on the faulty class, would directly address the false-negative risk.
3. **Capture complete cross-validation evidence for Modules 2 and 3.** Persisting and reporting the five-fold stratified cross-validation scores already computed by

Fault_type.py and Health_State_Classification.py, together with the internal test-split confusion matrix for Module 3, would close the evaluation completeness gap and provide direct evidence on estimator variance.

4. **Systematic hyperparameter search.** Replacing the literature informed but manually set hyperparameters for the Random Forest and XGBoost classifiers with a systematic, nested cross-validated search would test whether the accuracy gradient is sensitive to hyperparameter choice or is a more fundamental property of the dataset.
5. **Validation against physically instrumented, noisy sensor data.** The most significant direction for future work is to validate the pipeline against data captured from an actual plant controller and sensors mounted on a physical motor test rig under controlled fault injection, which would directly test the external validity concerns and provide the strongest possible evidence.
6. **Dynamic, editable maintenance knowledge base and continuous severity scoring.** Extending Module 4 to draw its cost, downtime and priority figures from a live asset-management or ERP system rather than a static CSV file, and extending Module 2 toward a continuous degradation index rather than five discrete severity bands as ways to bring the pipeline closer to a production grade maintenance decision support tool, consistent with the original project's own stated future improvement direction of ERP integration.

8.3 Closing Remarks

This project set out not to claim a new algorithm but to demonstrate, and fundamentally evaluate, a coherent, staged pipeline that connects raw multi-sensor motor data to a costed maintenance decision, a connection that the reviewed literature addresses only partially. The most valuable outcome of the evaluation was not the high accuracy achieved on two of the three learned modules, but the discipline of adjusting accuracy rather than accepting it, which surfaced concrete, specific and addressable next steps for making this pipeline ready for deployment.

References

1. **'Prediction of induction motor faults using machine learning'** :Abdulkareem, A., Anyim, T., Popoola, O., Abubakar, J. and Ayoade, A. (2024), Heliyon, 11(1), e41493.

2. **'Machine learning based fault classification for improved induction motor performance'** :Khan, Z.A., Raza, M.A., Masud, M.I., Jumani, T.A., Faheem, M. and Aman, M. (2025), PLOS ONE, 20(11), e0335367.

3. **'Predictive maintenance in the industry: A comparative study on deep learning-based remaining useful life estimation'** :Lorenti, L., Dalle Pezze, D., Andreoli, J., Masiero, C., Gentner, N., Yang, Y. and Susto, G.A. (2023), in Proceedings of the 2023 IEEE International Conference on Industrial Informatics (INDIN). IEEE.

4. **'Condition monitoring using machine learning: a review of theory, applications, and recent advances'** :Surucu, O., Gadsden, S.A. and Yawney, J. (2023), Expert Systems with Applications, 221, 119738.

5. **'Fault Analysis And Predictive Maintenance Of Induction Motor Using Machine Learning' :** Venkatesh, K. and Neethi, M. (2024), arXiv preprint arXiv:2409.09944.

Appendix A: Supporting Material

A.1 Model Hyperparameter Summary

Table A.1. Consolidated hyperparameter settings for the three learned classifiers.

Module	Algorithm	Key hyperparameters
1	KNeighborsClassifier	n_neighbors selected via sweep over k = 1 to 25 on held-out accuracy; StandardScaler applied to all features.
2	RandomForestClassifier	n_estimators = 200, max_depth = 12, min_samples_split = 5, min_samples_leaf = 2, random_state = 42.
3	XGBClassifier	objective = multi:softprob, n_estimators = 300, learning_rate = 0.05, max_depth = 6, subsample = 0.8, colsample_bytree = 0.8, eval_metric = mlogloss, random_state = 42.

A.2 Maintenance Knowledge Base

Table A.2. Full maintenance knowledge base used by Module 4, reproduced for reference.

Fault type	Priority	Action	Inspection	Cost (USD)	Hours	RUL (days)
none	-	No maintenance required. Continue monitoring.	Routine monthly inspection.	0	0h	180
imbalance	Medium	Perform rotor balancing.	As per knowledge base.	300	2h	35
bearing_wear	High	Replace motor bearings.	As per knowledge base.	450	4h	20
overload	High	Reduce operating load and inspect drive system.	As per knowledge base.	250	2h	15

Fault type	Priority	Action	Inspection	Cost (USD)	Hours	RUL (days)
phase_loss	Critical	Inspect electrical phases immediately.	As per knowledge base.	600	6h	5
insulation_failure	Critical	Replace insulation or rewind motor.	As per knowledge base.	900	8h	3

A.3 Reproducibility Notes

All three classifiers use a fixed random_state of 42 for train/test splitting and, where applicable, model initialization, to support reproducibility. All scripts read from and write to a fixed local project directory structure (Data/, models/, outputs/) as documented in the project README, and trained model artefacts persisted with joblib immediately after training so that the deployment test scripts can be run independently of the training scripts.

Appendix B: Code Repository

B.1 Repository URL

Repository: https://git.cs.bham.ac.uk/projects-2025-26/myr599

This is the GitLab project repository hosting the full codebase, dataset, trained models, firmware and supporting materials for this dissertation.

B.2 Repository Contents

The repository is organized into the directory structure summarized in Table B.1.

Table B.1. Repository directory structure and contents.

Directory / file	Contents
Data/	sensor_data.csv (primary 4,556-record training dataset), maintenance_rules.csv (maintenance knowledge base used by Module 4), mixed_data.csv (independent 128-record deployment-test file).
Data Samples/	Illustrative excerpts of the raw sensor data for quick inspection without loading the full dataset.
code/EDA/	Data_EDA.py, data quality checks and exploratory visualisation.
code/Module1-Binary Classification/	Health_detection.py, KNN training, k-selection sweep, ROC-AUC and confusion matrix (Module 1).
code/Module2-Health_state_classification/	Health_State_Classification.py, Random Forest training, cross-validation and confusion matrix (Module 2).
code/Module3-Fault_Type classification/	Fault_type.py, XGBoost training, cross-validation and confusion matrix (Module 3).
code/Module4-Cost Analysis/	Maintenance_Cost_Analysis.py, joins Module 3 predictions to the maintenance knowledge base and aggregates cost and downtime (Module 4).
Deployment codes/	firstmodel_Deploy.py, Health_state_Deploy.py and Fault_type_deployment_test.py, load the persisted models and score the independent deployment file.
firmware/	DHT11.ino, MPU6500.ino and Voltage.ino, Arduino sketches for the physical hardware proof of concept.
models/	Persisted model artefacts written by joblib: knn_model.pkl, scaler.pkl, Health_state_classifier.pkl, Health_state_label_encoder.pkl, xgboost_fault_classifier.pkl and fault_label_encoder.pkl.
outputs/images/	Confusion matrices, the ROC curve and the class-distribution and feature-distribution figures.

Directory / file	Contents
outputs/ (prediction files)	Module2_Predictions.csv, Module3_Predictions.csv and Maintenance_Recommendations.csv, the prediction and recommendation outputs.
README.md	Project overview, system architecture, dataset description and technology stack.
requirements.txt	Pinned Python package versions required to run the codebase.

B.3 Instructions to Run the Code

The scripts were developed and executed locally using absolute Windows file paths (for example, C:\MyWorkingDir\Workingpython\Msc Project\Data\sensor_data.csv). Before running the pipeline on a different machine, these hard-coded paths at the top of each script must be updated to point to the local copy of the Data/ and models/ directories described in Table B.1. Subject to that path update, the pipeline can be reproduced by following the six steps below.

1. **Clone the repository and install dependencies.** Clone the repository from the URL given in Section B.1, create a Python virtual environment, and install the pinned dependencies with pip install -r requirements.txt. The core dependencies are pandas, numpy, scikit-learn, xgboost, matplotlib, seaborn and joblib.
2. **Run the exploratory data analysis.** Run code/EDA/Data_EDA.py to reproduce the data quality checks and the class and feature distribution figures.
3. **Train Module 1 (binary health detection).** Run code/Module1-Binary Classification/Health_detection.py. This performs the k-selection sweep, trains the KNN classifier, and persists knn_model.pkl and scaler.pkl to the models/ directory.
4. **Train Modules 2 and 3 (severity and fault-type classification).** Run code/Module2-Health_state_classification/Health_State_Classification.py followed by code/Module3-Fault_Type classification/Fault_type.py. Each script trains its respective classifier, runs five-fold stratified cross-validation, and persists the trained model and its label encoder to models/.
5. **Run the deployment test scripts.** Run firstmodel_Deploy.py, Health_state_Deploy.py and Fault_type_deployment_test.py from the Deployment codes/ directory, in any order, against Data/mixed_data.csv. Each script loads its corresponding persisted model, scores the deployment file, prints the evaluation metrics, and writes its prediction CSV to outputs/.
6. **Generate maintenance recommendations.** Run code/Module4-Cost Analysis/Maintenance_Cost_Analysis.py, which reads Module3_Predictions.csv, joins it

to Data/maintenance_rules.csv, prints the per-fault and aggregate summaries, and writes Maintenance_Recommendations.csv to outputs/.

A note on run order: steps 3 and 4 (model training) must be completed before step 5 (deployment testing), since the deployment scripts load the .pkl artefacts produced by training; step 6 in turn depends on the Module3_Predictions.csv file produced by step 5. Running the scripts out of this order will raise a file not found error rather than silently producing incorrect results.

A note on the firmware. The three sketches in firmware/ (DHT11.ino, MPU6500.ino, Voltage.ino) are independent of the six steps above and are not required to reproduce the machine learning results. They were used for a physical hardware proof of concept on an ESP32 board, and can be opened directly in the Arduino IDE and flashed to an ESP32 for anyone wishing to reproduce that hardware testing.

Appendix C: Physical Hardware Implementation Evidence

This appendix reproduces the firmware source code and live serial monitor output captured directly from the Arduino IDE during the physical proof-of-concept hardware work carried out for this project. The screenshots are included here as primary evidence that the sensor readings discussed elsewhere in the report were acquired from real, running hardware rather than reconstructed after the fact, and are organized by sensor node.

C.1 Temperature and Humidity Node (DHT11)

Figure C.1 shows the complete DHT11.ino sketch as it appears in the Arduino IDE, running on the DOIT ESP32 DEVKIT V1 target selected in the board dropdown. Figure C.2 shows a captured excerpt of the resulting serial output, logged during a controlled rising-temperature test.

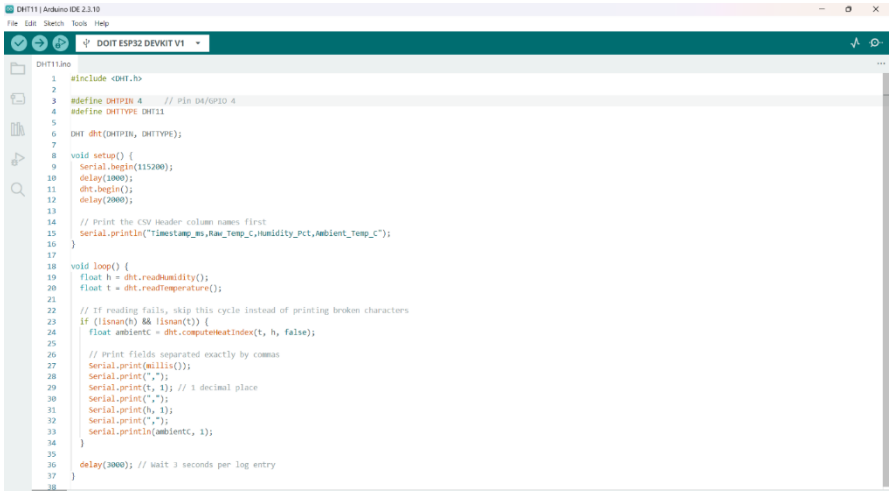


Figure C.1. DHT11.ino firmware, Arduino IDE 2.3.10, DOIT ESP32 DEVKIT V1 target.

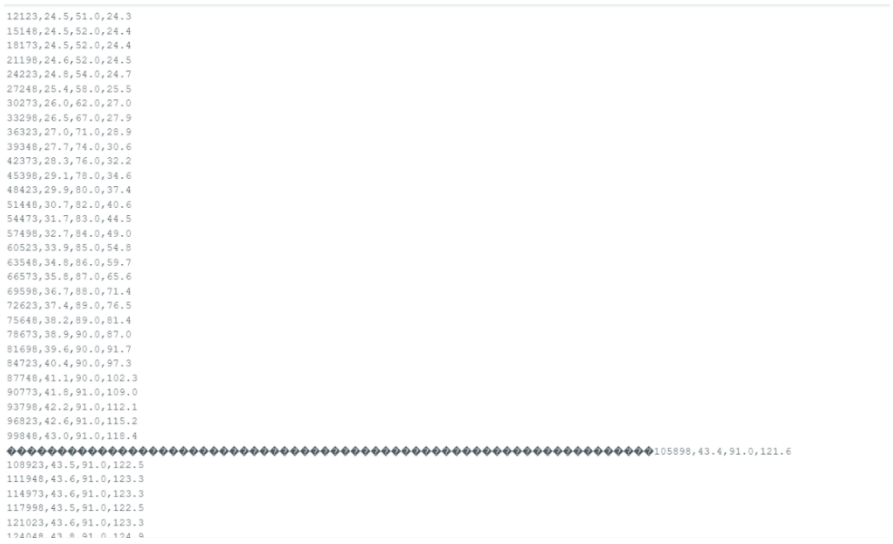


Figure C.2. Captured serial output from the DHT11 node during the thermal-load test.

C.2 Inertial Measurement Node (MPU-6500)

Figures C.3 and C.4 show the two halves of the MPU6500.ino sketch, covering the I2C wake-up sequence and register read on the first screen and the raw to engineering unit conversion and serial print statements on the second. Figure C.5 shows the live Serial Monitor while the board was being moved by hand, with both the accelerometer and gyroscope channels visibly responding to the motion in real time.

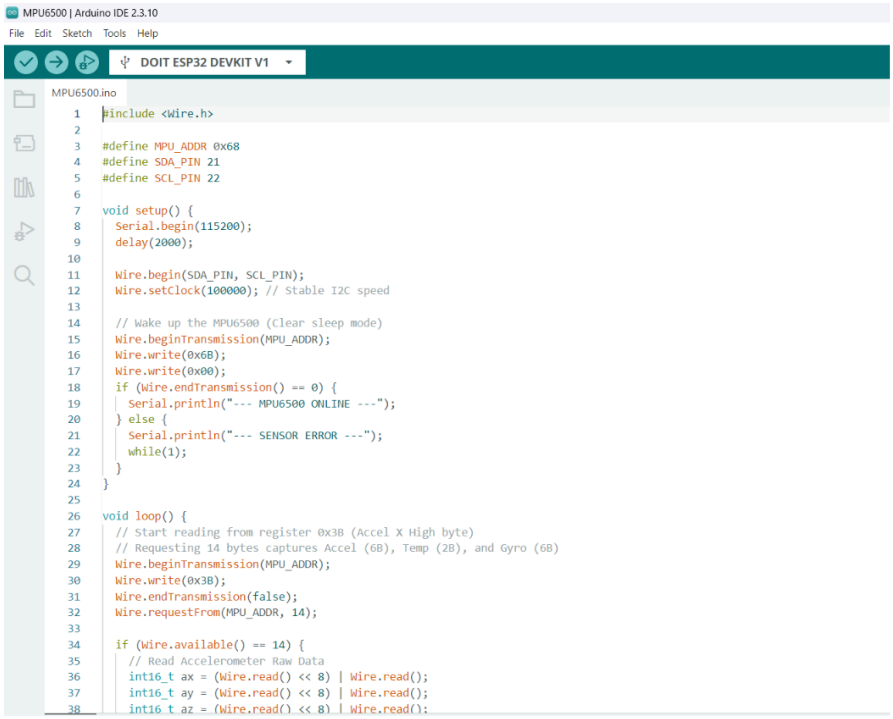


Figure C.3. MPU6500.ino firmware, part 1: I2C initialization and the 14-byte register burst read.



Figure C.4. MPU6500.ino firmware, part 2: unit conversion and serial output formatting.

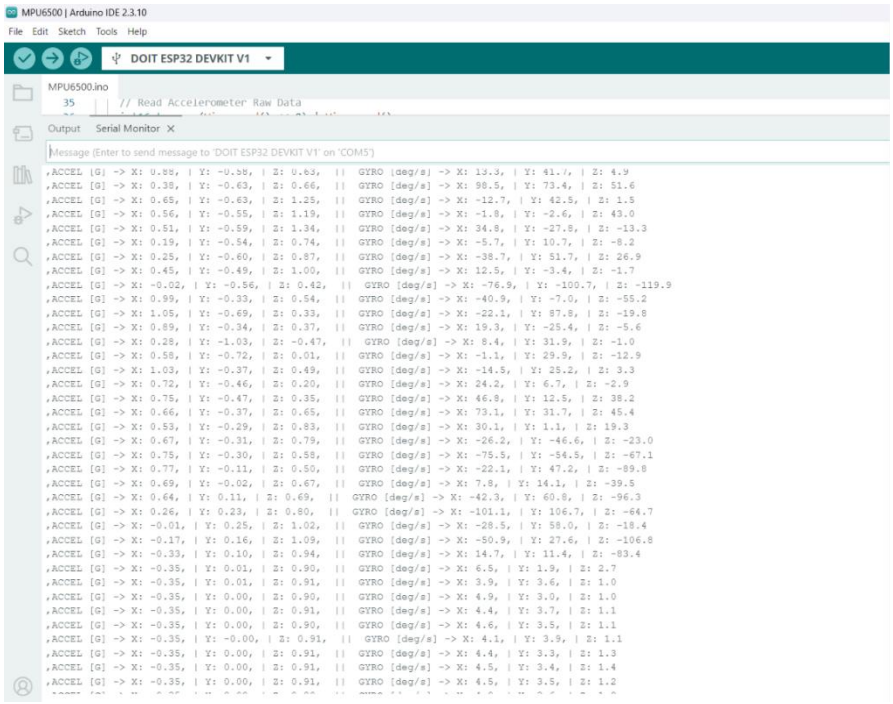


Figure C.5. Live Serial Monitor output from the MPU-6500 node while the board was moved by hand.

C.3 Voltage Sensing Node

Figures C.6 and C.7 show the Voltage.ino sketch, covering the resistive-divider constants and ADC-to-voltage conversion on the first screen and the CSV header and serial print statements on the second. Figure C.8 shows the resulting serial log, including a deliberate source

disconnection event used to confirm the sensing circuit behaves predictably when the input is removed.

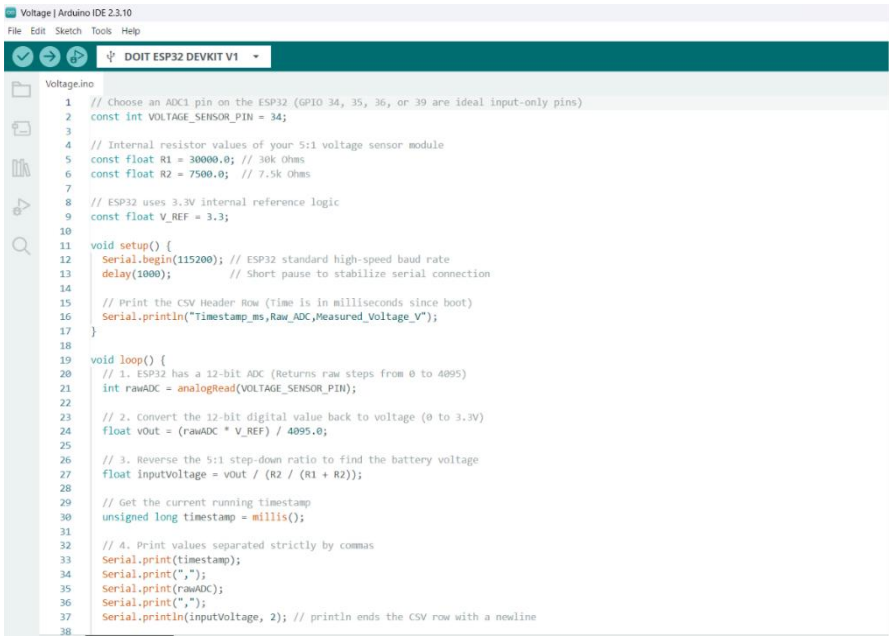


Figure C.6. Voltage.ino firmware, part 1: resistive-divider constants and the ADC-to-voltage conversion.

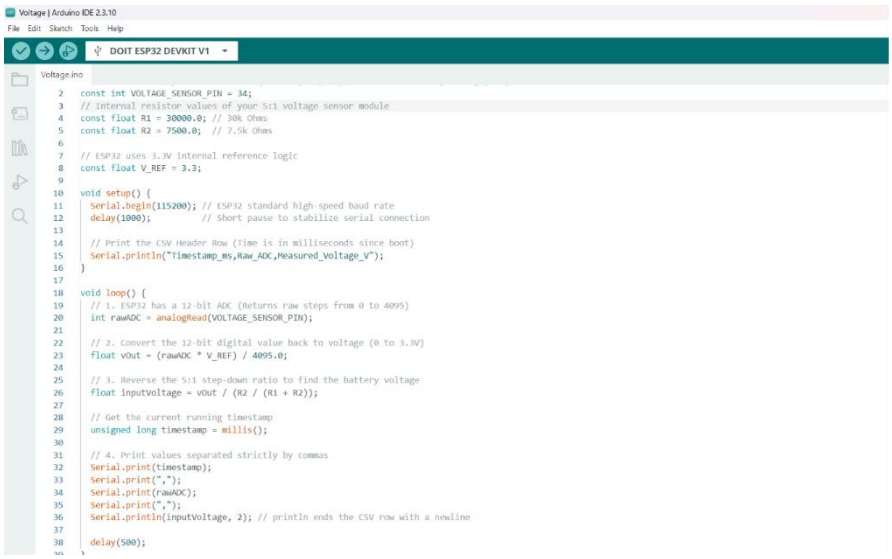


Figure C.7. Voltage.ino firmware, part 2: CSV header and serial print statements.

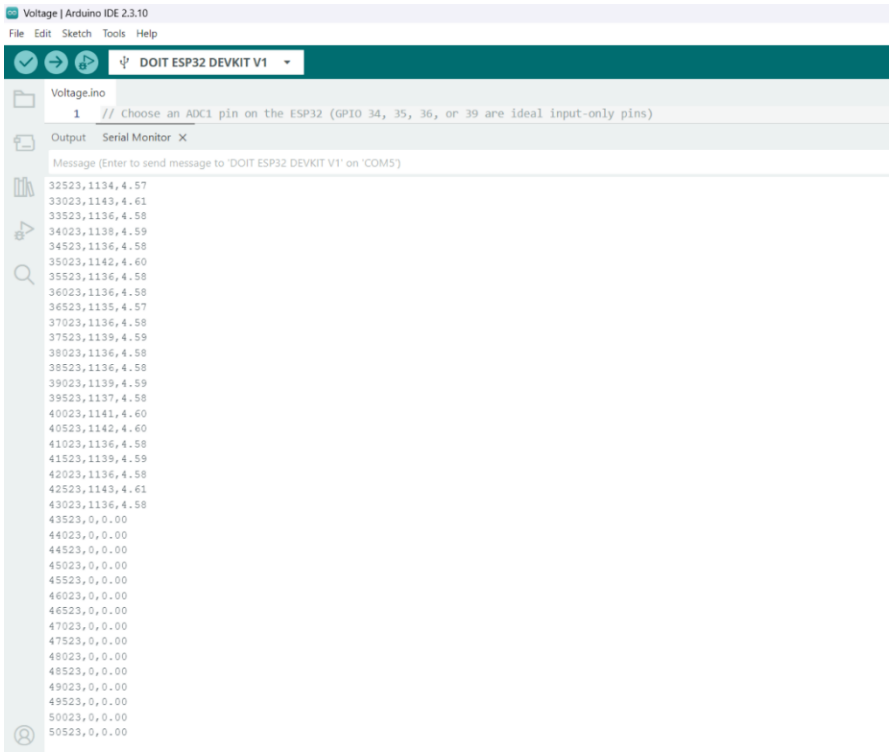


Figure C.8. Captured serial output from the voltage node, including the source-disconnection event.

C.4 Note on Provenance

All eight figures in this appendix are unedited screen captures taken directly from the Arduino IDE 2.3.10 environment, generated during the author's own hardware sessions on a DOIT ESP32 DEVKIT V1 board.